

Day06 课堂笔记

复习

挺好的,

建议新增每日抽查作业的流程, 一直不检查都开始想着浑水摸鱼了

冲冲冲

不定长参数的函数 在应用中不够熟练, 今天敲代码, 前一段时间学的一些定义忘记了, 需要多复习一下.

函数的引用还是经常会乱

```
12 print("%.3f" % num)
13 # f"{变量:.nf}"
14 print(f"{num:.3f}")
15 num1 = 1 #00000001
16 print('%6d' % num1)
17 print('%06d' % num1)
18 print(f'{num1:06d}')
```

```
3.140
3.140
      1
000001
000001
```

递归函数[理解]

递归: 函数自己嵌套调用自己.

已知：A 比 B 大 2 岁，B 比 C 大 2 岁，...，F 年龄为 18 岁，

求 A 的年龄。

知道第一个人的年龄是 18 岁

后边每个人比前边那个人大 2 岁

后边一个人的年龄 = 前边那个人的年龄 + 2

$\text{num} = (\text{num} - 1) + 2$

A (6)	B (5)	C (4)	D (3)	E (2)	F (1)
28	26	24	22	20	18

递归函数的形成条件:

1. 函数自己调用自己
2. 函数必须有一个终止条件

01-复习.py x 02-递归.py x

```
1 def get_age(num):
2     """ """
7     if num == 1:
8         return 18
9     # 求第 num 个人的年龄, 只需要num-1 这个人的年龄 + 2
10    age = get_age(num-1) + 2
11    return age
12
13
14 print(get_age(4))
```

get_age()

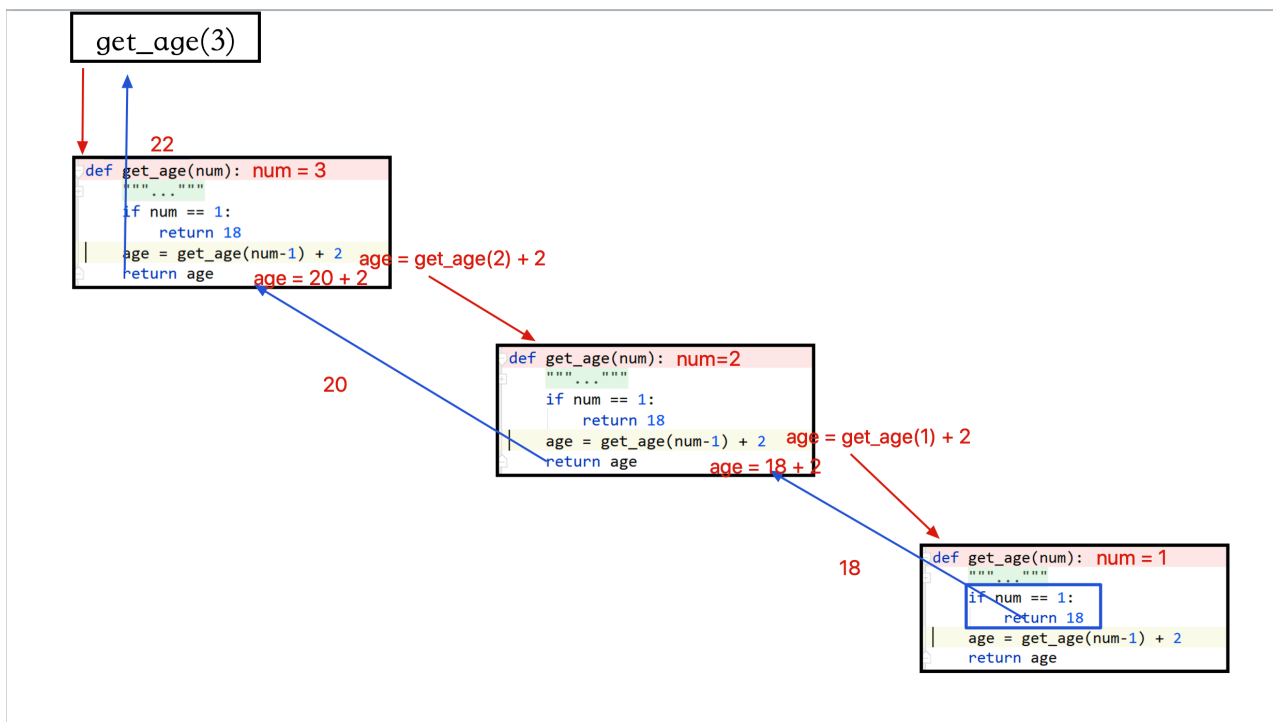
Debug: 02-递归

C:\Develop\python38\python.exe "C:\Program Files\JetBrains\PyCharm Community Edition 2020.1\plugins\pyt
pydev debugger: process 3396 is connecting

Connected to pydev debugger (build 201.6668.115)

24

Process finished with exit code 0



```
9      def func2(n):
10         if n == 1:
11             return 1
12         # 要求n! , 只需要(n-1)! * n
13         num = func2(n-1) * n
14         return num
15
16
17     # func(5)
18     print(func2(5))
```

匿名函数

使用 `lambda` 关键字定义的函数就是匿名函数

1 `lambda` 参数列表：表达式

1. 无参数无返回值

```
1  def 函数名():
2      函数代码
3
4  lambda:函数代码
```

2. 无参数有返回值

```
1 def 函数名():  
2     return 1 + 2  
3  
4 lambda: 1 + 2
```

3. 有参数无返回值

```
1 def 函数名(a, b):  
2     print(a, b)  
3  
4  
5 lambda a, b: print(a, b)
```

4. 有参数有返回值

```
1 def 函数名(a, b):  
2     return a + b  
3  
4  
5 lambda a, b: a + b
```

匿名函数的应用场景 -- 作为函数的参数使用

```
1 def my_calc(a, b, func):
2     """
3     进行四则运算
4     :param a: 第一个数据
5     :param b: 第二个数据
6     :param func: 函数,要进行的运算
7     :return: 运算的结果
8     """
9     print('其他的函数代码...')
10    num = func(a, b)
11    print(num)
12
13
14 def add(a, b):
15     return a + b
16
17
18 # 调用
19 my_calc(10, 20, add)
20 my_calc(10, 20, lambda a, b: a - b)
21 my_calc(10, 20, lambda a, b: a * b)
22 my_calc(10, 20, lambda a, b: a / b)
23 my_calc(10, 20, lambda a, b: a + b ** 2)
24
```

```
1 def my_calc(a, b, func): ① 函数定义，直接跳过，到 14 行
2     """
3     进行四则运算
4     :param a: 第一个数据
5     :param b: 第二个数据
6     :param func: 函数,要进行的运算
7     :return: 运算的结果
8     """
9     print('其他的函数代码...') ⑤
10    num = func(a, b) ⑥ func(10,20) 函数调用，即调用 add 函数，位置传参，
11    print(num)
12
13
14 def add(a, b): ② 函数定义，直接跳过不执行，到 19 行
15     return a + b ⑦ return 10 + 20
16
17
18 # 调用
19 my_calc(10, 20, add) ③ 函数调用，位置传参，回到第一行执行
20 my_calc(10, 20, lambda a, b: a - b)
21 my_calc(10, 20, lambda a, b: a * b)
22 my_calc(10, 20, lambda a, b: a / b)
23
24
```

④ a = 10
b = 20
func = add

lambda 函数作为函数参数(列表中字典排序)(扩展)

```
1 list1 = [{'name': 'd', 'age': 19}, {'name': 'b', 'age': 16}, {'name': 'a', 'age': 16}, {'name': 'c', 'age': 20}]
```

2

```
11 list1.sort() # 程序报错
12 print(list1)
```

列表中的数据是字典类型，字典如何比较大小，程序不知道，需要我们告诉程序，如何对字典比较大小

```
Run: C:\Develop\python38\python.exe C:/Users/n1/Desktop/43/day06/06-列表排序.py
[1, 1, 2, 3, 4, 5]
Traceback (most recent call last):
  File "C:/Users/n1/Desktop/43/day06/06-列表排序.py", line 11, in <module>
    list1.sort()
TypeError: '<' not supported between instances of 'dict' and 'dict'

Process finished with exit code 1
```

```
list1.sort(key=reverse)  # key形参，需要传递一个函数，指定规则，列表的数据根据什么排序
print(list1)  # 逆序，倒序排序list1
__setattr__(self, name, value)  # object
```

```
1  # 列表排序，列表中的数据要保持一致
2  my_list = [1, 3, 5, 4, 2, 1]
3  my_list.sort()
4  print(my_list)
5
6  list1 = [{'name': 'd', 'age': 19},
7           {'name': 'b', 'age': 16},
8           {'name': 'a', 'age': 16},
9           {'name': 'c', 'age': 20}]
10
11 # list1.sort() # 程序报错
12 # 匿名函数的形参是列表中的每一个数据
13 list1.sort(key=lambda x: x['name'])
14 print(list1)
15 list1.sort(key=lambda x: x['age'])
16 print(list1)
17
```

```
18 list2 = ['aghdd', 'bc', 'ghlj', 'def', ]
19 list2.sort()
20 print(list2)
21 # 需求：根据列表中字符串的长度，列表进行排序
22 # list2.sort(key=len)
23 list2.sort(key=lambda x: len(x))
24 print(list2)
```

列表推导式


```

1 # 列表推导式, 为了快速的生成一个列表
2 # 1. 变量 = [生成数据的规则 for 临时变量 in xxx]
3 # 每循环一次, 就会创建一个数据
4 my_list = [i for i in range(5)]
5 print(my_list) # [0, 1, 2, 3, 4]
6
7 my_list1 = ['hello' for i in range(5)]
8 print(my_list1)
9
10 my_list2 = [f'num:{i}' for i in my_list]
11 print(my_list2)
12 my_list3 = [i+i for i in range(5)]
13 print(my_list3)
14
15 # 2. 变量 = [生成数据的规则 for 临时变量 in xxx if xxx]
16 # 每循环一次, 并且if条件为True, 生成一个数据
17 my_list = [i for i in range(5) if i % 2 == 0]
18 print(my_list) # [0, 2, 4]
19
20 # 3. 变量 = [生成数据的规则 for 临时变量 in xxx for j in xxx]
21 # 第二个for 循环 循环一次, 生成一个数据
22 my_list4 = [(i,j) for i in range(3) for j in range(3)]
23 print(my_list4)

```

集合-set

```

5 my_set = {[1, 2]}
6 # 2. 集合是可变类型
7 # 3. 集合是无序的, (数据的添加顺序和输出顺序是否一致), 不支持下标操作
8 # 4. 集合中的数据不能有重复数据(去重)

```

集合中必须是不可变类型

字典中的 key 也必须是不可变类型
字典中的 key 是字符串和数字

Run: 08-集合 ×

```

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day06/08-集合.py
{1, (1, 2), 3.14, 'hello'} <class 'set'>
Traceback (most recent call last):
  File "C:/Users/nl/Desktop/43/day06/08-集合.py", line 5, in <module>
    my_set = {[1, 2]}
TypeError: unhashable type: 'list'

```

```

1  # 集合 set 定义使用 {}, {数据, 数据}
2  # 1. 集合中的数据必须是不可变类型
3  my_set = {1, 3.14, True, 'hello', (1, 2)}
4  print(my_set, type(my_set))
5  # my_set = {[1, 2]} # 代码报错
6  # 2. 集合是可变类型
7  my_set.remove(3.14)
8  print(my_set)
9  my_set.pop()
10 print(my_set)
11 my_set.add(100)
12 print(my_set)
13 # 修改数据 100 ---> 200
14 my_set.remove(100)
15 my_set.add(200)
16 print(my_set)
17 my_set.clear()
18 print(my_set) #
19 # 3. 集合是无序的, (数据的添加顺序和输出顺序是否一致), 不支持下标操作
20 # 4. 集合中的数据没有重复数据(去重)
21 my_list = [1, 2, 3, 4, 2, 5, 1, 0]
22 my_list = list(set(my_list))
23 print(my_list)
24 # 集合, 列表, 元组 三者之间可以互相转换

```

```

{((1, 2), 1, 'hello', 3.14)} <class 'set'>
{(1, 2), 1, 'hello'}
{1, 'hello'}
{1, 100, 'hello'}
{1, 200, 'hello'}
set()
[0, 1, 2, 3, 4, 5]

Process finished with exit code 0

```

激活 Windows

文件操作

文件的作用: 可以永久的保存数据.

文件在硬盘中存储的格式是二进制.

1. 打开文件
2. 读写文件
3. 关闭文件

读文件-r

```

1 # 1. 打开文件, 是文件从硬盘中存到内存中
2 # open(file, mode='r', encoding)
3 # file 要操作的文件名字, 类型是 str
4 # mode, 文件打开的方式, r(read) 只读打开, w(write) 只写打开 a(append) 追加打开
5 # encoding 文件的编码格式, 常见的编码格式有两种, 一种是gbk, 一种是utf-8
6 # 返回值, 文件对象, 后续所有的文件操作, 都需要通过这个文件对象进行
7
8 # 以只读的方式打开当前目录中, 1.txt 文件, 文件不存在会报错
9 f = open('1.txt', 'r')
10 # 2. 读文件 文件对象.read()
11 buf = f.read()
12 print(buf)
13 # 3. 关闭文件 文件.close() 将内存中三大文件同步到硬盘中
14 f.close()
15

```

写文件-w

File was loaded in the wrong encoding: 'UTF-8' Reload in 'GBK' Set project encoding to 'GBK' Reload in another encoding

```

1 hello world!
2 hello python!
3

```

1. open 函数打开文件, 没有指定文件的编码, windows 默认是 gbk,
2. write 函数 将 你好, 中国 写入文件中, 使用 gbk 编码写入
3. 在 pycharm 中双击打开文件, 默认使用的编码是 utf-8
4. 使用 utf-8 编码打开 gbk 编码的数据, 出现乱码

编码: 就是如何将中文汉字变为二进制, 或者如何将二进制转换为汉字

解决方案: 是两种方式的编码统一即可

方案一: open 打开文件的时候, 指定使用 utf-8 打开

方案二: pycharm 中使用 gbk 的方式打开

```

1 # 1. 打开文件 w 方式打开文件, 文件不存在, 会创建文件, 文件存在, 会覆盖清空原文件
2 f = open('a.txt', 'w', encoding='utf-8')
3 # 2. 写文件 文件对象.write(写入文件的内容)
4 f.write('hello world!\n')
5 f.write('hello python!\n')
6 f.write('你好, 中国!')
7 # 3. 关闭文件
8 f.close()
9

```

追加文件-a

```

1 # a 方式打开文件, 追加内容, 在文件的末尾写入内容
2 # 文件不存在, 会创建文件
3 # 注意点: 不管是a 方式打开文件, 还是 w 方式打开文件, 写内容, 都是使用 write()函数
4 f = open('b.txt', 'a', encoding='utf-8')
5 # f.write('hello world!\n')
6 f.write('111\n')
7 f.close()
8

```

